Technische Universität Bergakademie Freiberg
Institute of Numerical Mathematics and Optimization
Prof. Dr. O. Rheinbach
Dr. S. Köhler

Wintersemester 2026/27
28.10.2026

Praktikum Wissenschaftliches Rechnen / Scientific Programming Project

1. Exercise Sheet: Linear Regression in PYTHON with PYTORCH

Exercise 1
Given training data (X, Y) where $X \in \mathbb{R}^{n \times m}$ and $Y \in \mathbb{R}^n$, we want to find weights $w \in \mathbb{R}^m$ that minimize the mean squared error (MSE):

$$L(w) = \frac{1}{n} \sum_{i=1}^{n} (X_i \cdot w - Y_i)^2 = \frac{1}{n} \|Xw - Y\|^2 \tag{1}$$

The gradient of the loss with respect to w is:

$$\nabla_w L = \frac{2}{n} X^T (Xw - Y) \tag{2}$$

Gradient descent updates the weights iteratively:

$$w^{(t+1)} = w^{(t)} - \eta \nabla_w L \tag{3}$$

where η is the learning rate.
Create a Python script that implements four versions of linear regression, each with increasing levels of PyTorch abstraction. Implement also functions to create random datas.

Version 1: Pure NumPy Implementation

Implement everything from scratch to understand the fundamentals.
Requirements:

- Implement the forward pass: compute $\hat{Y} = Xw$.
- Implement MSE loss calculation.
- Implement gradient computation manually using the formula above.
- Implement gradient descent weight updates.
- Use NumPy arrays only.

Hints:

- Initialize weights to zeros.
- Print loss before training and every 100 epochs.
- Remember to use matrix operations for efficiency.

Version 2: PyTorch with Manual Optimization

Use PyTorch's autograd for gradient computation, but handle optimization manually.
Requirements:

- Convert NumPy arrays to PyTorch tensors.

- Create weights as a tensor with `requires_grad=True`.
- Use `.backward()` to compute gradients automatically.
- Manually update weights using `torch.no_grad()` context.
- Remember to zero gradients with `w.grad.zero_()` after eachupdate.

Key Differences from Version 1:

- No manual gradient function needed.
- Use `lo.backward()` instead of computing gradients manually.
- Must zero gradients explicitly.

Version 3: PyTorch with Optimizer

Use PyTorch's optimizer to handle weight updates.
Requirements:

- Keep autograd for gradient computation
- Use `nn.MSELoss()` for loss calculation.
- Create `torch.optim.SGD` optimizer.
- Use `optimizer.step()` for weight updates.
- Use `optimizer.zero_grad()` to zero gradients.

Key Differences from Version 2:

- Optimizer handles the weight update logic.
- Cleaner code with `optimizer.step()` and `optimizer.zero_grad()`.

Version 4: PyTorch Neural Network Module

Use PyTorch's `nn.Linear` layer for a complete neural network approach.
Requirements:

- Reshape Y to shape `(n_samples, 1)` for compatibility with `nn.Linear` output.
- Use `nn.Linear(input_size, output_size)` to create the model.
- Note: `nn.Linear` includes a bias term by default.
- Pass `model.parameters()` to the optimizer.
- Call model as a function: `y_pred = model(X)`.

Important Notes

- `nn.Linear` includes bias, so input X should NOT have a bias column.
- Return `model.parameters()`.
- This version is closest to how you'd implement real neural networks.